



ДИСЦИПЛИНА	<u>Разработка приложений на языке Котлин</u> (полное наименование дисциплины без сокращений)
ИНСТИТУТ	<u>информационных технологий</u>
КАФЕДРА	<u>информационных технологий в атомной энергетике</u> полное наименование кафедры
ВИД УЧЕБНОГО МАТЕРИАЛА	<u>Лекция</u> (в соответствии с пп 1-11)
ПРЕПОДАВАТЕЛЬ	<u>Золотухин Святослав Александрович</u> (фамилия, имя, отчество)
СЕМЕСТР	<u>5 семестр 2025 – 2026 учебный год</u> (указать семестр обучения, учебный год)



Лекция 8. Разработка приложений на языке Котлин

ФИО преподавателя: Золотухин Святослав Александрович

e-mail: zolotuhin@mirea.ru



Создание последовательности

Функция `sequenceOf`

Последовательности представляют интерфейс `Sequence<T>`

```
val people = sequenceOf("Andrew", "Alice", "Kirill") //тип  
Sequence<String>
```

```
println(people.joinToString()) // Andrew, Alice, Kirill
```



Метод `asSequence` коллекций

Можно создать последовательность из объекта `Iterable`, используя метод `asSequence()`

```
val employees = listOf("Andrew", "Alice", "Kirill") //  
объект List<String>
```

```
val people = employees.asSequence()           //тип  
Sequence<String>
```

```
println(people.joinToString()) // Andrew, Alice, Kirill
```



Функция `generateSequence`

Третий способ создания последовательности представляет функция `generateSequence`

```
var num = 0
```

```
val nums = generateSequence{ number += 2; num }
```

```
println(nums.take(5).joinToString()) // получаем  
первые 5 элементов последовательности: 2, 4, 6, 8,  
10
```



```
var num = 0
```

```
val nums = generateSequence{ number += 2; if(num >  
8) null else number}
```

```
println(nums.toString()) // 2, 4, 6, 8
```

```
val numbers = generateSequence(5){ if(it == 25) null  
else it + 5}
```

```
println(numbers.toString()) // 5, 10, 15, 20, 25
```



Функция `sequence`

Можно генерировать элементы последовательности с помощью функций **`yield()`** и **`yieldAll()`**

```
val numbers = sequence {  
    yield(1)  
    yield(5)  
    yield(7)  
}  
  
println(numbers.joinToString()) // 1, 5, 7
```



Функция sequence

Можно создать бесконечную последовательность:

```
val numbers = sequence {
```

```
    var start = 0
```

```
    while(true) yield(start++)
```

```
}
```

```
println(numbers.take(5).joinToString()) // 0, 1, 2, 3, 4
```




Функция sequence

```
val personal = sequence {  
    val data = listOf("Kate", "Ann", "Alice")  
    yieldAll(data)  
}  
println(personal.joinToString()) // Kate, Ann, Alice
```



Перебор последовательности

```
val people = sequenceOf("Andrew", "Kirill", "Peter")  
for(person in people) println(person)
```



Операции с последовательностями

Интерфейс **Sequence** предоставляет ряд функций, который позволяют производить различные операции с последовательностями:

all(predicate: (T) -> Boolean): Boolean

возвращает true, если все элементы соответствуют предикату, который передается в функцию в качестве параметра



Операции с последовательностями

any(): Boolean

возвращает true, если последовательность содержит хотя бы один элемент

average(): Double

возвращает среднее значение для числовой последовательности типов Byte, Int, Short, Long, Float, Double



Последовательности VS Коллекции

1. Промежуточные результаты:

Коллекция Iterable: Каждая операция при применении к коллекции возвращает промежуточную коллекцию.

Последовательность Sequence: Весь набор операций выполняется только при запросе конечного результата обработки, избегая создания промежуточных коллекций.

2. Порядок применения операций:

Коллекция Iterable: Каждая операция применяется последовательно ко всем элементам, создавая промежуточные коллекции.

Последовательность Sequence: Весь набор операций применяется отдельно к каждому элементу, избегая промежуточных коллекций.

3. Эффективность:

Коллекция Iterable: Подходит для небольших объемов данных и малого количества операций.

Последовательность Sequence: Повышает производительность при выполнении набора операций, особенно для больших наборов данных, избегая создания промежуточных коллекций.



Последовательности

Фильтрация (по условию)

```
fun main(){  
    var people = sequenceOf("Tom", "Sam", "Mike", "Bob", "Alice")  
    people = people.filter{it.length == 3} // получаем значения с длиной в 3 символа  
    println(people.joinToString()) // Tom, Sam, Bob  
    var employees = listOf(  
        Person("Tom", 37),  
        Person("Bob", 41),  
        Person("Sam", 25)  
    )  
    employees = employees.filter{it.age > 30} // получаем всех Person, у которых age > 30  
    println(employees.joinToString()) // Person(name=Tom, age=37), Person(name=Bob, age=41)  
}
```

```
data class Person(val name: String, val age: Int)
```



Последовательности

Фильтрация (по индексу)

```
val people = listOf("Tom", "Mike", "Sam", "Bob", "Alice")  
// получаем значения с длиной в 3 символа на четных индексах  
val filtered = people.filterIndexed{ index, s -> (index % 2 == 0) && (s.length == 3)}  
println(filtered) // [Tom, Sam]
```



Последовательности

Фильтрация (по типу)

```
fun main(){  
    val people = listOf(  
        Person("Tom"), Employee("Bob"),  
        Person("Sam"), Employee("Mike")  
    )  
    // получаем только элементы типа Employee  
    val employees = people.filterIsInstance<Employee>();  
    println(employees) // [Bob, Mike]  
}  
  
open class Person(val name: String){  
    override fun toString(): String = name  
}  
  
class Employee(name: String): Person(name)
```




Последовательности

Фильтрация (по null)

```
fun main(){  
    val people = listOf(Person("Tom"), null, Person("Sam"), null)  
    println(people)    // [Tom, null, Sam, null]  
    val filtered = people.filterNotNull()  
    println(filtered)  // [Tom, Sam]  
}  
  
open class Person(val name: String){  
    override fun toString(): String = name  
}
```



Последовательности

Проверка соответствия элементов условию

$\text{all}(\text{predicate: (T)} \rightarrow \text{Boolean}): \text{Boolean}$

$\text{any}(\text{predicate: (T)} \rightarrow \text{Boolean}): \text{Boolean}$

$\text{none}(\text{predicate: (T)} \rightarrow \text{Boolean}): \text{Boolean}$



Последовательности

Проверка соответствия элементов условию

```
val people = listOf("Tom", "Kate", "Sam", "Alice", "Bob")
```

```
// all
```

```
println(people.all{it.length == 3})    // false
```

```
println(people.all{it.length != 10})    // true
```

```
// none
```

```
println(people.none{it.length == 3})    // false
```

```
println(people.none{it.length == 2})    // true
```

```
// any
```

```
println(people.any{it.length == 3})    // true
```

```
println(people.any{it.length == 10})    // false
```



Последовательности

Трансформации

```
fun main(){  
    val people = listOf(Person("Tom"), Person("Sam"),  
        Person("Bob"))  
  
    val names = people.map { it.name } // возвращаем имя  
    каждого пользователя  
    println(names) // [Tom, Sam, Bob]  
}  
  
class Person(val name: String)
```



Последовательности

Группировка

```
fun main(){  
    val people = listOf(Person("Tom"), Person("Sam"),  
        Person("Bob"))  
  
    val names = people.map { it.name } // возвращаем имя  
    каждого пользователя  
    println(names) // [Tom, Sam, Bob]  
}  
  
class Person(val name: String)
```



Последовательности

Сортировка

Для сортировки коллекции/последовательности применяются функции **sorted()** (сортировка по возрастанию) и **sortedDescending()** (сортировка по убыванию).

Для встроенных базовых типов действует следующая логика:

- Числа сравниваются как в математике, исходя из их значения
- Символы (Char) и строки (String) сравниваются, исходя из лексикографического порядка, то есть "Tom" больше, чем "Alice", потому что первый символ - Т располагается в алфавите после символа А.



Последовательности

Сортировка

Если определяются свои типы и нужно, чтобы их объекты также можно было отсортировать, то в этом случае следует реализовать интерфейс **Comparable**.

Его функция **compareTo()** должна определять логику сравнения. В качестве параметра она принимает объект, который сравнивается с текущим.

В качестве результата возвращается число. Если текущий объект равен переданному объекту, то функция должна вернуть 0. Если текущий объект больше переданного объекта, то возвращается положительное число, если меньше - то отрицательное



Последовательности

Сортировка

sortedWith и интерфейс **Comparator**:

- **sortedWith** в Kotlin используется для сортировки списка с использованием пользовательского компаратора.
- Интерфейс **Comparator** предоставляет метод **compare**, который позволяет определить порядок элементов.



Последовательности

Сортировка

sortedWith и интерфейс Comparator:

```
val numbers = listOf(3, 5, 2, 1, 4)
```

```
val sortedNumbers = numbers.sortedWith(compareBy  
{ it }) // Сортировка по возрастанию
```

```
println(sortedNumbers) // [1, 2, 3, 4, 5]
```



Последовательности

Сортировка

Сортировка на основе критерия:

Функция `sortedBy` в Kotlin используется для сортировки списка на основе определенного критерия.



Последовательности

Сортировка

```
data class Person(val name: String, val age: Int)  
val people = listOf(Person("Tom", 30), Person("Alice",  
25), Person("Bob", 35))  
  
val sortedPeople = people.sortedBy { it.age } //  
Сортировка по возрасту  
println(sortedPeople) // [Alice(25), Tom(30), Bob(35)]
```



Последовательности

Сортировка

reversed:

Функция `reversed` в Kotlin используется для изменения порядка элементов в списке на противоположный.

shuffle:

Функция `shuffle` в Kotlin используется для перемешивания элементов в списке.



Последовательности

Агрегатные операции

Минимальное и максимальное значение
(**minOrNull()** и **maxOrNull()**)

Для получения среднего значения применяется
функция **average()**

Для получения суммы числовых значений
применяется функция **sum()**

Для получения количества элементов в
коллекции/последовательности применяется
функция **count()**



Последовательности

Агрегатные операции

Функция **reduce()** сводит все значения потока к одному значению

```
val numbers = listOf(1, 2, 3, 4, 5)
```

```
val reducedValue = numbers.reduce{ a, b -> a + b }
```

```
println(reducedValue) // 15
```



Последовательности

Сложение, вычитание и объединение коллекций

```
fun main(){  
    val people = listOf("Tom", "Bob", "Sam")  
    // добавляем один объект  
    val result1 = people.plus("Alice")  
    println(result1) // [Tom, Bob, Sam, Alice]  
  
    val employees = listOf("Mike", "Kate")  
    // добавляем коллекцию объектов  
    val result2 = people.plus(employees)  
    println(result2) // [Tom, Bob, Sam, Mike, Kate]  
}
```



Последовательности

Сложение, **вычитание** и объединение коллекций

```
fun main(){  
    val people = listOf("Tom", "Bob", "Sam", "Kate")  
    // вычитаем один объект  
    val result1 = people.minus("Bob")  
    println(result1) // [Tom, Sam, "Kate"]  
  
    val employees = listOf("Mike", "Kate")  
    // вычитаем коллекцию  
    val result2 = people.minus(employees)  
    println(result2) // [Tom, Bob, Sam]  
}
```




Последовательности

Сложение, вычитание и **объединение** коллекций

```
val english = listOf("red", "blue", "green")
```

```
val russian = listOf("красный", "синий", "зеленый")
```

```
val words = english.zip(russian)
```

```
for(word in words)
```

```
    println("${word.first}: ${word.second}")
```



Последовательности

Сложение, вычитание и **объединение** коллекций

```
val dictionary = listOf("red", "blue", "green")  
    .zip(listOf("красный", "синий", "зеленый"))  
val words = dictionary.unzip()  
println(words.first) // [red, blue, green]  
println(words.second) // [красный, синий, зеленый]
```



Получение части элементов

slice()

```
val people = listOf("Tom", "Bob", "Sam", "Kate", "Alice",  
"Mike")
```

```
println(people.slice(1..3))    // [Bob, Sam, Kate]
```

```
println(people.slice(0..5 step 2)) // [Tom, Sam, Alice]
```

```
println(people.slice(listOf(1, 3, 5, 1))) // [Bob, Kate, Mike, Bob]
```



Получение части элементов

Take и takeWhile

```
val people = listOf("Tom", "Bob", "Sam", "Kate", "Alice",  
"Mike")
```

```
println(people.take(3))    // ["Tom", "Bob", "Sam"]
```

```
val people = listOf("Tom", "Bob", "Sam", "Kate", "Alice",  
"Mike")
```

```
// получаем три последних элемента
```

```
println(people.takeLast(3)) // ["Kate", "Alice", "Mike"]
```



Получение части элементов

Take и takeWhile

```
val people = listOf("Tom", "Sam", "Kate", "Bob", "Alice",  
"Mike")
```

```
val part = people.takeWhile{it.length == 3}
```

```
println(part)    // ["Tom", "Sam"]
```



Получение части элементов

drop и dropWhile

```
val people = listOf("Tom", "Sam", "Kate", "Bob", "Alice",  
"Mike")
```

```
val part = people.drop(3) // пропускаем первые 3 элемента  
println(part)    // [Bob, Alice, Mike]
```



Получение части элементов

drop и dropWhile

```
val people = listOf("Tom", "Sam", "Kate", "Bob", "Alice",  
"Mike")
```

```
val part = people.dropWhile{it.length == 3}
```

```
println(part)    // [Kate, Bob, Alice, Mike]
```



Получение части элементов

Разделение на части, функция `chunked()`

```
val people = listOf("Tom", "Sam", "Kate", "Bob", "Alice",  
"Mike")
```

```
val parts = people.chunked(3)
```

```
println(parts)    // [[Tom, Sam, Kate], [Bob, Alice, Mike]]
```




Получение отдельных элементов

Получение элемента по индексу

```
val people = listOf("Tom", "Sam", "Kate", "Bob", "Alice")  
// получаем элемент по индексу 1  
println(people.elementAt(1))    // Sam
```



Получение отдельных элементов

Получение элемента по индексу

```
val people = listOf("Tom", "Sam", "Kate", "Bob", "Alice")  
println(people.elementAtOrNull(1))    // Sam  
println(people.elementAtOrNull(6))    // null
```



Получение отдельных элементов

Получение элемента по индексу

```
val people = listOf("Tom", "Sam", "Kate", "Bob", "Alice")
```

```
println(people.elementAtOrElse(1){"Undefined"})    // Sam
```

```
println(people.elementAtOrElse(6){"Undefined"})    //  
Undefined
```

```
println(people.elementAtOrElse(8){"Index $it out of bounds"})  
// Index 8 out of bounds
```



Получение отдельных элементов

Получение элемента по условию

```
val people = listOf("Tom", "Sam", "Kate", "Bob", "Alice")  
println(people.first())    // Tom  
println(people.last())     // Alice
```



Получение отдельных элементов

Выбор случайного элемента

```
val people = listOf("Tom", "Sam", "Kate", "Bob", "Alice")  
println(people.random())
```



Спасибо за внимание!